

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	R SARVESH	Reg. No.:	192521143
Section / Batch:	B.TECH IT	Date:	23/7/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

- Q1.** File Search System
A file search system uses a divide-and-conquer approach by reducing the search space by half in each step. Define logarithmic time complexity and explain how halving affects performance. Determine the time complexity of the algorithm. Analyze its efficiency for large datasets. Interpret the result using asymptotic notation.
- Q2.**

Autonomous Driving Data Processing System

An autonomous driving system processes sensor data using the recurrence $T(n) = 2T(n/2) + n^2$. Apply the Master Theorem to determine the time complexity of the algorithm. Clearly identify the parameters a , b , and $f(n)$, and justify the appropriate case selection. Analyze how the performance of the system is affected as the volume of sensor data increases. Interpret the result using asymptotic notation and explain its impact on real-time decision-making.

Code of Conduct

I R SARVESH (192521143) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R SARVESH

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING
CSA0620-DESIGN AND ANALYSIS OF ALGORITHM

NAME: R SARVESH

REG. NO: 192521143

DEPT.: B.TECH INFORMATION TECHNOLOGY

CO1 ASSESSMENT TOOL 1

1. File Search System: A file search system uses a divide-and-conquer approach by reducing the search space by half in each step. Define logarithmic time complexity and explain how halving affects performance. Determine the time complexity of the algorithm. Analyze its efficiency for large datasets. Interpret the result using asymptotic notation.

A file search system reduces the search space by half in each step. This represents a **binary search approach**.

At every step:

- The input size becomes half.
- Only one half is searched further.
- The process continues until the file is found or the search space becomes empty.

Logarithmic Time Complexity

The recurrence is:

$$T(n) = T(n/2) + 1$$

where:

- n = number of files
- $n/2$ = remaining search space after each step
- 1 = comparison operation

Using Master Theorem:

$$a=1, b=2, f(n)=1$$

$$n^{\log b^a} = n^{\log 2^1} = 1$$

Both are equal:

$$f(n) = n^{\log b^a}$$

Therefore:

$$T(n) = \Theta(\log n)$$

Efficiency Analysis

For large datasets:

- Linear search checks every file $\rightarrow O(n)$
- Binary search repeatedly halves the dataset $\rightarrow O(\log n)$

Example:

For 1,000,000 files:

Linear search:

1,000,000 comparisons

Binary search:

$$\log_2(1000000) \approx 20$$

Only around **20 comparisons** are required.

Asymptotic Interpretation

$$T(n) = O(\log n)$$

The algorithm scales efficiently because the execution time increases very slowly even when the dataset size grows.

C Program: File Search System Using Binary Search

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0;
    int high = n - 1;

    while(low <= high)
    {
        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;

        else if(arr[mid] < key)
            low = mid + 1;

        else
            high = mid - 1;
    }

    return -1;
}
```

```
int main()
{
    int n, key;

    printf("Enter number of files: ");
    scanf("%d",&n);

    int files[n];

    printf("Enter sorted file IDs:\n");
```

```
for(int i=0;i<n;i++)
    scanf("%d",&files[i]);

printf("Enter file ID to search: ");
scanf("%d",&key);

int result = binarySearch(files,n,key);

if(result != -1)
    printf("File found at position %d\n",result);

else
    printf("File not found\n");

return 0;
}
```

Output:

```
Output
Enter number of files: 8
Enter sorted file IDs:
10 20 30 40 50 60 70 80
Enter file ID to search: 50
File found at position 4

=== Code Execution Successful ===
```

2. Autonomous Driving Data Processing System An autonomous driving system processes sensor data using the recurrence $T(n) = 2T(n/2) + n^2$. Apply the Master Theorem to determine the time complexity of the algorithm. Clearly identify the parameters a , b , and $f(n)$, and justify the appropriate case selection. Analyze how the performance of the system is affected as the volume of sensor data increases. Interpret the result using asymptotic notation and explain its impact on real-time decision-making.

Answer:

An autonomous vehicle processes sensor data using the recurrence:

$$T(n)=2T(n/2)+ n^2$$

Where:

- Recursive processing handles divided sensor data.
- Combining results requires polynomial computation.

Master Theorem Analysis

General recurrence:

$$T(n)=aT(n/b)+f(n)$$

Given:

$$T(n)=2T(n/2)+ n^2$$

Parameters:

Parameter	Value
a	2
b	2
f(n)	n^2

Calculate:

$$n^{\log_b a}$$

Substitute values:

$$n^{\log 2^2} = n$$

Now compare:

$$f(n) = n^2$$

with

$$n^{\log b^a} = n$$

Since:

$$n^2 > n$$

Master Theorem **Case 3 applies.**

Therefore:

$$T(n) = \Theta(n^2)$$

Performance Analysis

As sensor data increases:

Example:

Sensor Data	Processing Cost
n=100	10,000 operations
n=1000	1,000,000 operations
n=10000	100,000,000 operations

The processing cost increases quadratically.

Impact:

- Larger sensor inputs require significantly more computation.
- Real-time decisions may experience delays.
- Optimization of the combine phase is important.

C Program: Autonomous Sensor Data Processing

```
#include <stdio.h>

int processSensorData(int n)
{
    if(n <= 1)
        return 1;

    int left = processSensorData(n/2);
    int right = processSensorData(n/2);

    int combineCost = n*n;

    return left + right + combineCost;
}

int main()
{
    int n;

    printf("Enter sensor data size: ");
    scanf("%d",&n);

    int result = processSensorData(n);

    printf("Total Processing Cost: %d\n",result);

    return 0;
}
```

Output:

```
Output
Enter sensor data size: 8
Total Processing Cost: 120

=== Code Execution Successful ===
```